Index No.: 230280L
Name: J. A. P. R. Jayasinghe
Github link: https://github.com/pemitharj/EN3160_Assignment_3

Question 1

In [5]:
```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

batch_size = 50
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuff
testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size, shuffle
```

(a) Add a middle layer (100 nodes, sigmoid)

In [ ]:
```
Din, H, K = 3*32*32, 100, 10
std = 1e-2

# Layer 1: input -> 100 hidden nodes
w1 = torch.randn(Din, H) * std
b1 = torch.zeros(H)
# Layer 2: 100 hidden nodes -> 10 outputs
w2 = torch.randn(H, K) * std
b2 = torch.zeros(K)

def sigmoid(z):
    return 1.0 / (1.0 + torch.exp(-z))

def forward(x):
    h = sigmoid(x.mm(w1) + b1)   # Middle layer with sigmoid activation
    scores = h.mm(w2) + b2       # Output layer
    return h, scores
```

(b) Cross entropy loss

In [ ]:
```
def softmax(z):
    z = z - z.max(dim=1, keepdim=True).values
    e = torch.exp(z)
    return e / e.sum(dim=1, keepdim=True)

def cross_entropy(scores, y_onehot):
    N = scores.shape[0]
    p = softmax(scores)
    loss = -(1.0 / N) * torch.sum(y_onehot * torch.log(p + 1e-12))
    return loss, p
```

```
def backward(x, h, p, y_onehot):
    N = x.shape[0]
    dscores = (p - y_onehot) / N   # Gradient of cross-entropy + softmax
    dw2 = h.t().mm(dscores)
    db2 = dscores.sum(dim=0)
    dh = dscores.mm(w2.t())
    dz1 = dh * h * (1 - h)   # Sigmoid derivative
    dw1 = x.t().mm(dz1)
    db1 = dz1.sum(dim=0)
    return dw1, db1, dw2, db2
```

(c) Run 10 epochs, report train and test accuracy

In [ ]:

```
epochs = 10
lr = 5e-2
lr_decay = 0.9
loss_history = []

for t in range(epochs):
    running_loss = 0.0
    for inputs, labels in trainloader:
        N = inputs.shape[0]
        x = inputs.view(N, -1)
        y = nn.functional.one_hot(labels, K).float()

        h, scores = forward(x)
        loss, p = cross_entropy(scores, y)
        loss_history.append(loss.item())
        running_loss += loss.item()

        dw1, db1, dw2, db2 = backward(x, h, p, y)
        w1 -= lr * dw1
        b1 -= lr * db1
        w2 -= lr * dw2
        b2 -= lr * db2

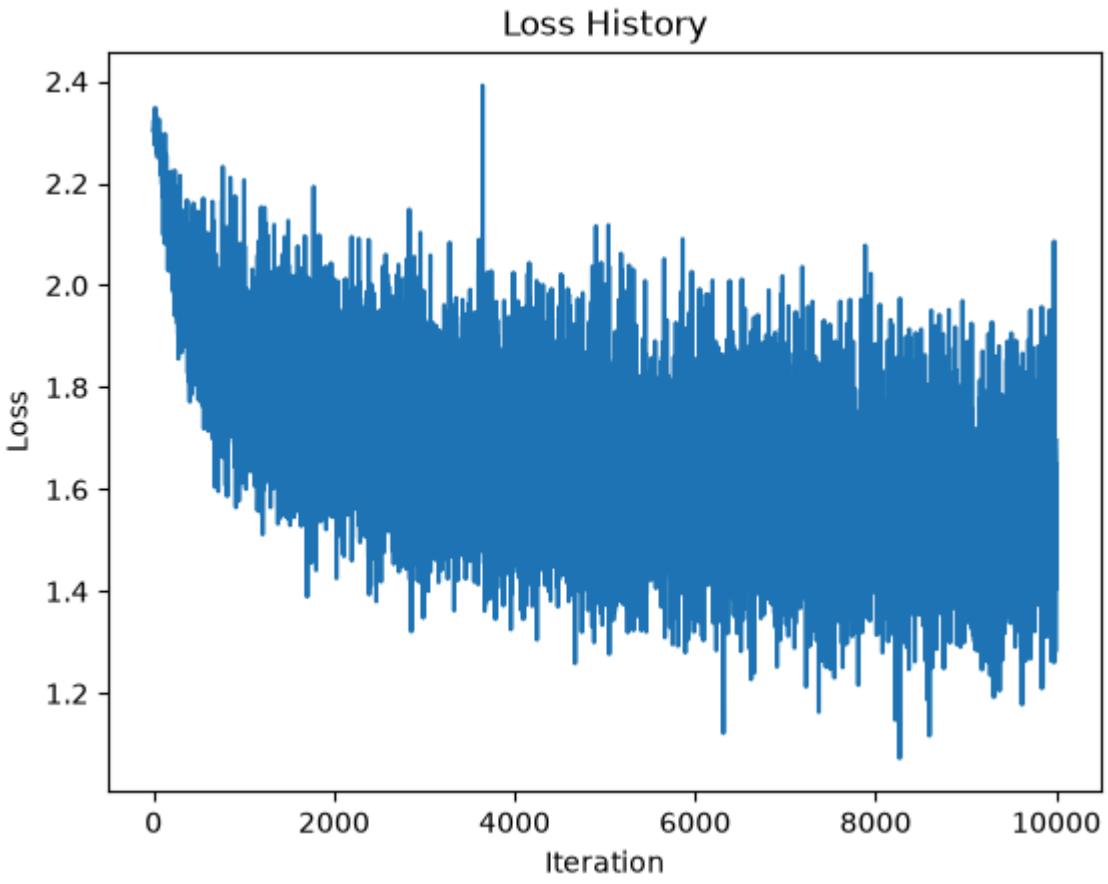
    print(f"Epoch {t+1}/{epochs}, Loss: {running_loss/len(trainloader):.4f}")
    lr *= lr_decay

# Loss plot
plt.plot(loss_history)
plt.title("Loss History"); plt.xlabel("Iteration"); plt.ylabel("Loss")
plt.show()

# Accuracy
def accuracy(loader):
    correct, total = 0, 0
    with torch.no_grad():
        for inputs, labels in loader:
            x = inputs.view(inputs.shape[0], -1)
            _, scores = forward(x)
            correct += (scores.argmax(dim=1) == labels).sum().item()
            total += labels.size(0)
    return 100 * correct / total

print(f"Training accuracy: {accuracy(trainloader):.2f}%")
print(f"Test accuracy: {accuracy(testloader):.2f}%")
```

```
Epoch 1/10, Loss: 1.9845
Epoch 2/10, Loss: 1.7925
Epoch 3/10, Loss: 1.7251
Epoch 4/10, Loss: 1.6836
Epoch 5/10, Loss: 1.6543
Epoch 6/10, Loss: 1.6300
Epoch 7/10, Loss: 1.6098
Epoch 8/10, Loss: 1.5918
Epoch 9/10, Loss: 1.5768
Epoch 10/10, Loss: 1.5629
```




```
Training accuracy: 46.47%
Test accuracy: 44.91%
```

Analysis

The listing used `lr = 2e-6` and `std = 1e-5`, which were too small for a network with a hidden layer and cross-entropy loss (the weights would barely move in 10 epochs). I increased them and compared two learning rates; `std = 1e-2` was used in both runs.

Run	lr	Epoch-1 loss	Epoch-10 loss	Train acc	Test acc
1	0.01	2.21	1.79	36.95%	37.08%
2	0.05	1.98	1.56	46.47%	44.91%

Discussion

- Increasing the learning rate from 0.01 to 0.05 improved test accuracy from 37.08% to 44.91% and reduced the final loss from 1.79 to 1.56, indicating better learning within 10 epochs.

- The second run maintained a lower loss throughout training, starting at 1.98 compared with 2.21 in the first run. This suggests that the higher learning rate enabled faster convergence through larger parameter updates.
- The loss was still decreasing at epoch 10, suggesting that additional training might improve performance. However, further experiments are needed to determine whether more epochs or a different learning rate would be beneficial.
- The second run achieved 46.47% training accuracy and 44.91% test accuracy, a small gap of 1.56 percentage points. This indicates that there is no strong evidence of overfitting at this stage.
- The model achieved only around 45% test accuracy partly because flattening images into one-dimensional vectors loses their spatial structure. Unlike dense networks, CNNs can learn local patterns such as edges, shapes, and neighbouring pixels using local connections and shared weights, often with fewer parameters.
- Cross-entropy loss combined with softmax provides a useful gradient for classification, simplifying the output-layer gradient to (p-y), where (p) is the predicted probability vector and (y) is the one-hot target vector. In contrast, MSE combined with softmax can produce smaller gradients for saturated predictions, potentially slowing learning when predictions are incorrect.

Question 2

In [9]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))])

train_ds = torchvision.datasets.MNIST(root='./data', train=True, download=True,
test_ds = torchvision.datasets.MNIST(root='./data', train=False, download=True,
train_loader = torch.utils.data.DataLoader(train_ds, batch_size=64, shuffle=True
test_loader = torch.utils.data.DataLoader(test_ds, batch_size=1000, shuffle=Fals

def evaluate(model, loader):
    model.eval()
    criterion = nn.CrossEntropyLoss(reduction='sum')
    total_loss, correct, n = 0.0, 0, 0
    preds, labs = [], []
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
```

```
            out = model(x)
            total_loss += criterion(out, y).item()
            p = out.argmax(dim=1)
            correct += (p == y).sum().item()
            n += y.size(0)
            preds.append(p.cpu()); labs.append(y.cpu())
    return total_loss / n, 100 * correct / n, torch.cat(preds).numpy(), torch.ca

def train_model(model, epochs=10, lr=1e-3):
    model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    hist = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}
    for ep in range(epochs):
        model.train()
        run_loss, correct, n = 0.0, 0, 0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            out = model(x)
            loss = criterion(out, y)
            loss.backward()
            optimizer.step()
            run_loss += loss.item() * y.size(0)
            correct += (out.argmax(dim=1) == y).sum().item()
            n += y.size(0)
        te_loss, te_acc, _, _ = evaluate(model, test_loader)
        hist["train_loss"].append(run_loss / n)
        hist["train_acc"].append(100 * correct / n)
        hist["test_loss"].append(te_loss)
        hist["test_acc"].append(te_acc)
        print(f"Epoch {ep+1}/{epochs} | train loss {run_loss/n:.4f}, acc {100*cc
              f"| test loss {te_loss:.4f}, acc {te_acc:.2f}%")
    return hist
```

```
100.0%
100.0%
100.0%
100.0%
```

(a) LeNet-5

In [ ]:

```
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.ReLU(),    # 28x28 -> 2
            nn.MaxPool2d(2),                                         # 14x14
            nn.Conv2d(6, 16, kernel_size=5), nn.ReLU(),              # 10x10
            nn.MaxPool2d(2))                                         # 5x5
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16*5*5, 120), nn.ReLU(),
            nn.Linear(120, 84), nn.ReLU(),
            nn.Linear(84, 10))

    def forward(self, x):
        return self.classifier(self.features(x))

torch.manual_seed(0)
```

```
lenet = LeNet5()
print("LeNet-5 params:", sum(p.numel() for p in lenet.parameters()))
hist_lenet = train_model(lenet, epochs=10)

_, tr_acc, _, _ = evaluate(lenet, torch.utils.data.DataLoader(train_ds, batch_si
print(f"LeNet-5 final train acc: {tr_acc:.2f}%, test acc: {hist_lenet['test_acc'
```

```
LeNet-5 params: 61706
Epoch 1/10 | train loss 0.2272, acc 92.99% | test loss 0.0746, acc 97.63%
Epoch 2/10 | train loss 0.0625, acc 97.99% | test loss 0.0458, acc 98.38%
Epoch 3/10 | train loss 0.0453, acc 98.56% | test loss 0.0495, acc 98.42%
Epoch 4/10 | train loss 0.0355, acc 98.87% | test loss 0.0388, acc 98.73%
Epoch 5/10 | train loss 0.0299, acc 99.01% | test loss 0.0335, acc 98.91%
Epoch 6/10 | train loss 0.0255, acc 99.17% | test loss 0.0350, acc 98.84%
Epoch 7/10 | train loss 0.0217, acc 99.29% | test loss 0.0317, acc 99.04%
Epoch 8/10 | train loss 0.0180, acc 99.42% | test loss 0.0342, acc 99.03%
Epoch 9/10 | train loss 0.0167, acc 99.42% | test loss 0.0318, acc 99.02%
Epoch 10/10 | train loss 0.0157, acc 99.52% | test loss 0.0331, acc 99.08%
LeNet-5 final train acc: 99.71%, test acc: 99.08%
```

Explanation

- LeNet-5 achieved 99.71% training accuracy and 99.08% test accuracy with 61,706 parameters.
- Test accuracy increased rapidly, reaching 97.63% in the first epoch and stabilizing around 99% from epoch 7.
- Training loss continued to decrease, while test loss levelled off near 0.03. The small accuracy gap of 0.63 percentage points suggests mild overfitting.

(b) Modern small CNN

In [11]:
```
class ModernCNN(nn.Module):
    def __init__(self):
        super().__init__()
        def block(cin, cout):
            return nn.Sequential(
                nn.Conv2d(cin, cout, kernel_size=3, padding=1),
                nn.BatchNorm2d(cout), nn.ReLU(),
                nn.MaxPool2d(2))
        self.features = nn.Sequential(block(1, 32), block(32, 64), block(64, 128
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(0.5),
            nn.Linear(128*3*3, 10))

    def forward(self, x):
        return self.classifier(self.features(x))

torch.manual_seed(0)
modern = ModernCNN()
print("Modern CNN params:", sum(p.numel() for p in modern.parameters()))
hist_modern = train_model(modern, epochs=10)

_, tr_acc, _, _ = evaluate(modern, torch.utils.data.DataLoader(train_ds, batch_s
print(f"Modern CNN final train acc: {tr_acc:.2f}%, test acc: {hist_modern['test_
```

```
Modern CNN params: 104650
Epoch 1/10 | train loss 0.1415, acc 95.67% | test loss 0.0451, acc 98.57%
Epoch 2/10 | train loss 0.0556, acc 98.25% | test loss 0.0340, acc 98.86%
Epoch 3/10 | train loss 0.0428, acc 98.69% | test loss 0.0247, acc 99.23%
Epoch 4/10 | train loss 0.0363, acc 98.83% | test loss 0.0308, acc 98.96%
Epoch 5/10 | train loss 0.0309, acc 99.00% | test loss 0.0248, acc 99.14%
Epoch 6/10 | train loss 0.0267, acc 99.10% | test loss 0.0272, acc 99.15%
Epoch 7/10 | train loss 0.0243, acc 99.19% | test loss 0.0357, acc 98.79%
Epoch 8/10 | train loss 0.0226, acc 99.27% | test loss 0.0254, acc 99.18%
Epoch 9/10 | train loss 0.0190, acc 99.38% | test loss 0.0227, acc 99.28%
Epoch 10/10 | train loss 0.0176, acc 99.40% | test loss 0.0218, acc 99.30%
Modern CNN final train acc: 99.70%, test acc: 99.30%
```

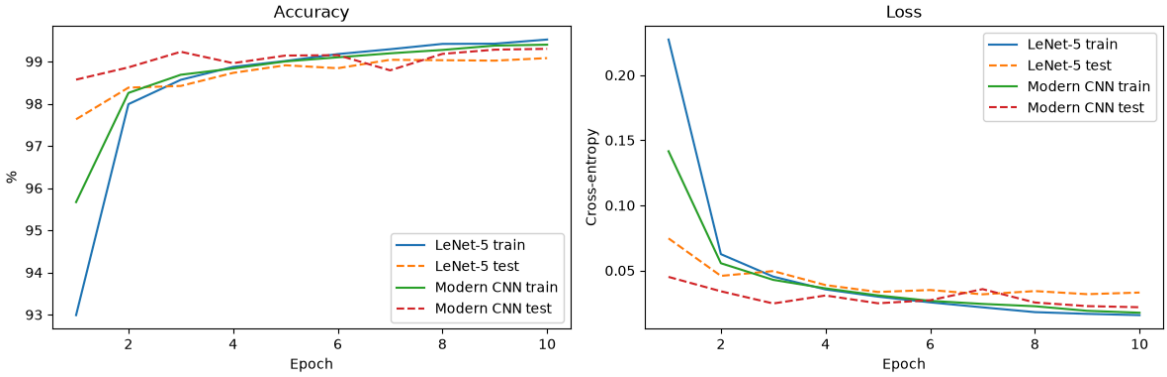
Explanation

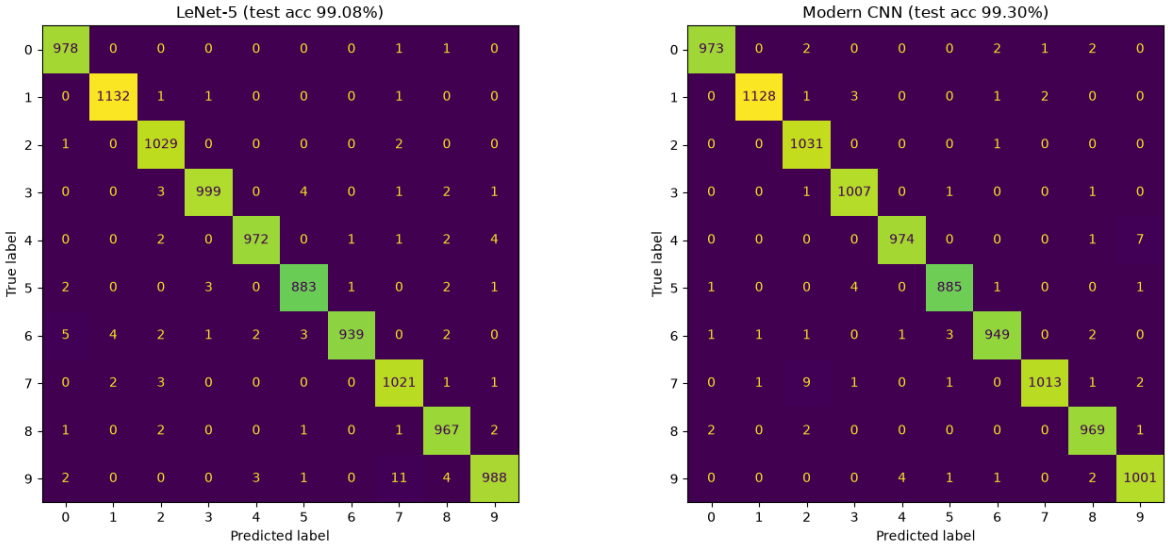
- The modern CNN achieved 99.70% training accuracy and 99.30% test accuracy with 104,650 parameters.
- It converged faster than LeNet-5, achieving 98.57% test accuracy in epoch 1 and 99.23% by epoch 3. It also achieved a lower final test loss (0.0218 vs. 0.0331).
- The smaller train–test accuracy gap of 0.40 percentage points suggests good generalization. Dropout and Batch Normalization may have contributed to stable performance.
- Test accuracy fluctuated slightly, dipping to 98.79% at epoch 7, before recovering. Overall, the modern CNN achieved slightly better accuracy with approximately 1.7 times more parameters. Since MNIST is relatively simple, both models performed close to the maximum achievable accuracy.

(c) per-epoch plots + Confusion matrices

In [12]:
```
# Accuracy and loss curves
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ep = range(1, 11)
for name, h in [("LeNet-5", hist_lenet), ("Modern CNN", hist_modern)]:
    ax[0].plot(ep, h["train_acc"], label=f"{name} train")
    ax[0].plot(ep, h["test_acc"], "--", label=f"{name} test")
    ax[1].plot(ep, h["train_loss"], label=f"{name} train")
    ax[1].plot(ep, h["test_loss"], "--", label=f"{name} test")
ax[0].set_title("Accuracy"); ax[0].set_xlabel("Epoch"); ax[0].set_ylabel("%"); a
ax[1].set_title("Loss"); ax[1].set_xlabel("Epoch"); ax[1].set_ylabel("Cross-entr
plt.tight_layout(); plt.show()

# Confusion matrices on the test set
fig, ax = plt.subplots(1, 2, figsize=(14, 6))
for a, (name, m) in zip(ax, [("LeNet-5", lenet), ("Modern CNN", modern)]):
    _, acc, preds, labs = evaluate(m, test_loader)
    cm = confusion_matrix(labs, preds)
    ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(ax=a, colorbar=Fal
    a.set_title(f"{name} (test acc {acc:.2f}%)")
plt.tight_layout(); plt.show()
```





In [18]:
```python
# Checking differences between the 2 confusion matrices
for name, m in [("LeNet-5", lenet), ("Modern CNN", modern)]:
    _, _, preds, labs = evaluate(m, test_loader)
    cm = confusion_matrix(labs, preds)
    np.fill_diagonal(cm, 0)
    top = np.dstack(np.unravel_index(np.argsort(cm.ravel())[::-1][:3], cm.shape))
    print(name)
    for t, p in top:
        print(f"  true {t} predicted as {p}: {cm[t, p]} times")
```

```
LeNet-5
  true 9 predicted as 7: 11 times
  true 6 predicted as 0: 5 times
  true 6 predicted as 1: 4 times
Modern CNN
  true 7 predicted as 2: 9 times
  true 4 predicted as 9: 7 times
  true 5 predicted as 3: 4 times
```

Explanation

- Learning curves: Both models showed increasing accuracy and decreasing loss during training. The modern CNN generally performed slightly better, while dropout reduced its effective training accuracy during training.
- Confusion matrices: Both models classified almost all digits correctly (strong diagonal). LeNet-5's most frequent errors were 9 predicted as 7 (11 times), 6 as 0 (5 times) and 6 as 1 (4 times). The modern CNN's most frequent errors were 7 predicted as 2 (9 times), 4 as 9 (7 times) and 5 as 3 (4 times). The two models did not make the same top mistakes, so their errors are not identical. The pairs involved (4/9,

5/3, 7/2) are digits that can look alike in some handwriting styles, which is a likely reason, though we did not inspect the individual images. Even the most common error happened only about 11 times out of 10,000 test images, so both models are very accurate.

Comparison of the two models

Model	Parameters	Train acc	Test acc	Final test loss
LeNet-5	61,706	99.71%	99.08%	0.0331
Modern CNN	104,650	99.70%	99.30%	0.0218

The modern CNN reached slightly higher test accuracy and a lower test loss, with a smaller train/test gap (0.40 vs 0.63 points). It also converged faster in the early epochs. The gain is small because MNIST is easy and both models are already close to 100% accuracy.

Question 3

In [13]:
```
import os, time, copy, zipfile, urllib.request
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import lr_scheduler
import torchvision
from torchvision import datasets, models, transforms
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Download + extract hymenoptera
if not os.path.exists("data/hymenoptera_data"):
    os.makedirs("data", exist_ok=True)
    url = "https://download.pytorch.org/tutorial/hymenoptera_data.zip"
    urllib.request.urlretrieve(url, "data/hymenoptera_data.zip")
    with zipfile.ZipFile("data/hymenoptera_data.zip") as z:
        z.extractall("data")

data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])}

image_datasets = {x: datasets.ImageFolder(os.path.join("data/hymenoptera_data",
                  for x in ['train', 'val']}
dataloaders = {x: torch.utils.data.DataLoader(image_datasets[x], batch_size=4,
                                              shuffle=(x == 'train'), num_worker
               for x in ['train', 'val']}
```

```
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
class_names = image_datasets['train'].classes
print(dataset_sizes, class_names)

def train_model(model, criterion, optimizer, scheduler, epochs=25):
    hist = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}
    start = time.time()
    for ep in range(epochs):
        for phase in ['train', 'val']:
            model.train() if phase == 'train' else model.eval()
            run_loss, correct = 0.0, 0
            for x, y in dataloaders[phase]:
                x, y = x.to(device), y.to(device)
                optimizer.zero_grad()
                with torch.set_grad_enabled(phase == 'train'):
                    out = model(x)
                    loss = criterion(out, y)
                    if phase == 'train':
                        loss.backward()
                        optimizer.step()
                run_loss += loss.item() * x.size(0)
                correct += (out.argmax(1) == y).sum().item()
            if phase == 'train':
                scheduler.step()
            hist[f"{phase}_loss"].append(run_loss / dataset_sizes[phase])
            hist[f"{phase}_acc"].append(100 * correct / dataset_sizes[phase])
        print(f"Epoch {ep+1}/{epochs} | train loss {hist['train_loss'][-1]:.4f},
              f"| val loss {hist['val_loss'][-1]:.4f}, acc {hist['val_acc'][-1]:
    hist["time"] = time.time() - start
    print(f"Training time: {hist['time']:.0f}s")
    return hist
```

```
{'train': 244, 'val': 153} ['ants', 'bees']
```

(a) Fine-tuning

In [14]:

```
torch.manual_seed(0)
model_ft = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
model_ft.fc = nn.Linear(model_ft.fc.in_features, 2)
model_ft = model_ft.to(device)

criterion = nn.CrossEntropyLoss()
optimizer_ft = optim.SGD(model_ft.parameters(), lr=0.001, momentum=0.9)
scheduler_ft = lr_scheduler.StepLR(optimizer_ft, step_size=7, gamma=0.1)

hist_ft = train_model(model_ft, criterion, optimizer_ft, scheduler_ft, epochs=25
print(f"Fine-tuning: final val acc {hist_ft['val_acc'][-1]:.2f}%, best {max(hist
```

```
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to C:\Us
ers\ASUS/.cache\torch\hub\checkpoints\resnet18-f37072fd.pth
100.0%
```

```
Epoch 1/25 | train loss 0.5840, acc 69.26% | val loss 0.2130, acc 92.16%
Epoch 2/25 | train loss 0.5240, acc 75.82% | val loss 0.1968, acc 92.16%
Epoch 3/25 | train loss 0.6799, acc 70.90% | val loss 0.2635, acc 92.81%
Epoch 4/25 | train loss 0.4558, acc 81.97% | val loss 0.3133, acc 84.97%
Epoch 5/25 | train loss 0.4837, acc 79.51% | val loss 0.2740, acc 91.50%
Epoch 6/25 | train loss 0.3275, acc 86.89% | val loss 0.2116, acc 94.12%
Epoch 7/25 | train loss 0.3787, acc 82.38% | val loss 0.3049, acc 88.24%
Epoch 8/25 | train loss 0.3659, acc 84.84% | val loss 0.2311, acc 92.81%
Epoch 9/25 | train loss 0.3740, acc 83.61% | val loss 0.2384, acc 92.81%
Epoch 10/25 | train loss 0.3355, acc 88.52% | val loss 0.2073, acc 91.50%
Epoch 11/25 | train loss 0.3853, acc 81.56% | val loss 0.1883, acc 92.16%
Epoch 12/25 | train loss 0.2560, acc 89.34% | val loss 0.1919, acc 91.50%
Epoch 13/25 | train loss 0.3305, acc 85.66% | val loss 0.1945, acc 91.50%
Epoch 14/25 | train loss 0.2489, acc 89.75% | val loss 0.2202, acc 91.50%
Epoch 15/25 | train loss 0.2707, acc 88.93% | val loss 0.2669, acc 90.85%
Epoch 16/25 | train loss 0.2580, acc 88.52% | val loss 0.1961, acc 92.81%
Epoch 17/25 | train loss 0.2252, acc 90.98% | val loss 0.2051, acc 92.81%
Epoch 18/25 | train loss 0.3400, acc 84.02% | val loss 0.2357, acc 90.85%
Epoch 19/25 | train loss 0.1696, acc 92.21% | val loss 0.1938, acc 92.81%
Epoch 20/25 | train loss 0.2638, acc 88.52% | val loss 0.1966, acc 92.16%
Epoch 21/25 | train loss 0.3005, acc 86.07% | val loss 0.2095, acc 92.81%
Epoch 22/25 | train loss 0.2539, acc 89.34% | val loss 0.1875, acc 90.85%
Epoch 23/25 | train loss 0.1944, acc 92.62% | val loss 0.2035, acc 92.81%
Epoch 24/25 | train loss 0.3432, acc 84.02% | val loss 0.1914, acc 92.16%
Epoch 25/25 | train loss 0.2527, acc 88.93% | val loss 0.1925, acc 92.81%
Training time: 1296s
Fine-tuning: final val acc 92.81%, best 94.12%
```

Explanation

- The model achieved **92.81% final validation accuracy**, with a best accuracy of 94.12% at epoch 6 and 88.93% training accuracy.
- Validation accuracy reached 92% in the first epoch due to the pretrained ImageNet weights, then fluctuated between 91% and 94%, with a dip to 84.97% at epoch 4.
- Training accuracy gradually increased, while validation loss remained around 0.19–0.20, suggesting limited further improvement.
- Training took **1,296 seconds** because gradients were computed throughout the network.

(b) ResNet18 as a feature extractor

In [15]:
```
torch.manual_seed(0)
model_fe = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
for p in model_fe.parameters():
    p.requires_grad = False   # Freeze the backbone
model_fe.fc = nn.Linear(model_fe.fc.in_features, 2)   # Only this layer trains
model_fe = model_fe.to(device)

optimizer_fe = optim.SGD(model_fe.fc.parameters(), lr=0.001, momentum=0.9)
scheduler_fe = lr_scheduler.StepLR(optimizer_fe, step_size=7, gamma=0.1)

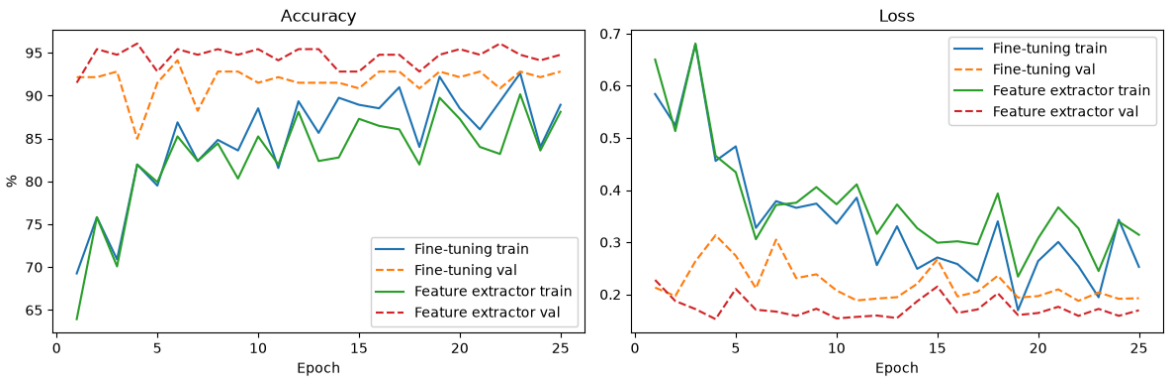
hist_fe = train_model(model_fe, criterion, optimizer_fe, scheduler_fe, epochs=25
print(f"Feature extractor: final val acc {hist_fe['val_acc'][-1]:.2f}%, best {ma
```

```
Epoch 1/25 | train loss 0.6502, acc 63.93% | val loss 0.2276, acc 91.50%
Epoch 2/25 | train loss 0.5130, acc 75.82% | val loss 0.1872, acc 95.42%
Epoch 3/25 | train loss 0.6809, acc 70.08% | val loss 0.1718, acc 94.77%
Epoch 4/25 | train loss 0.4656, acc 81.97% | val loss 0.1525, acc 96.08%
Epoch 5/25 | train loss 0.4343, acc 79.92% | val loss 0.2104, acc 92.81%
Epoch 6/25 | train loss 0.3057, acc 85.25% | val loss 0.1705, acc 95.42%
Epoch 7/25 | train loss 0.3712, acc 82.38% | val loss 0.1669, acc 94.77%
Epoch 8/25 | train loss 0.3756, acc 84.43% | val loss 0.1587, acc 95.42%
Epoch 9/25 | train loss 0.4056, acc 80.33% | val loss 0.1724, acc 94.77%
Epoch 10/25 | train loss 0.3726, acc 85.25% | val loss 0.1539, acc 95.42%
Epoch 11/25 | train loss 0.4108, acc 81.97% | val loss 0.1568, acc 94.12%
Epoch 12/25 | train loss 0.3158, acc 88.11% | val loss 0.1594, acc 95.42%
Epoch 13/25 | train loss 0.3723, acc 82.38% | val loss 0.1546, acc 95.42%
Epoch 14/25 | train loss 0.3269, acc 82.79% | val loss 0.1867, acc 92.81%
Epoch 15/25 | train loss 0.2990, acc 87.30% | val loss 0.2149, acc 92.81%
Epoch 16/25 | train loss 0.3017, acc 86.48% | val loss 0.1643, acc 94.77%
Epoch 17/25 | train loss 0.2957, acc 86.07% | val loss 0.1711, acc 94.77%
Epoch 18/25 | train loss 0.3934, acc 81.97% | val loss 0.2022, acc 92.81%
Epoch 19/25 | train loss 0.2340, acc 89.75% | val loss 0.1603, acc 94.77%
Epoch 20/25 | train loss 0.3074, acc 87.30% | val loss 0.1642, acc 95.42%
Epoch 21/25 | train loss 0.3669, acc 84.02% | val loss 0.1762, acc 94.77%
Epoch 22/25 | train loss 0.3266, acc 83.20% | val loss 0.1587, acc 96.08%
Epoch 23/25 | train loss 0.2447, acc 90.16% | val loss 0.1721, acc 94.77%
Epoch 24/25 | train loss 0.3395, acc 83.61% | val loss 0.1588, acc 94.12%
Epoch 25/25 | train loss 0.3142, acc 88.11% | val loss 0.1694, acc 94.77%
Training time: 372s
Feature extractor: final val acc 94.77%, best 96.08%
```

Results

In [16]:
```
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ep = range(1, 26)
for name, h in [("Fine-tuning", hist_ft), ("Feature extractor", hist_fe)]:
    ax[0].plot(ep, h["train_acc"], label=f"{name} train")
    ax[0].plot(ep, h["val_acc"], "--", label=f"{name} val")
    ax[1].plot(ep, h["train_loss"], label=f"{name} train")
    ax[1].plot(ep, h["val_loss"], "--", label=f"{name} val")
ax[0].set_title("Accuracy"); ax[0].set_xlabel("Epoch"); ax[0].set_ylabel("%"); a
ax[1].set_title("Loss"); ax[1].set_xlabel("Epoch"); ax[1].legend()
plt.tight_layout(); plt.show()

print(f"{'Method':<20}{'Train acc':>10}{'Val acc':>10}{'Time (s)':>10}")
for name, h in [("Fine-tuning", hist_ft), ("Feature extractor", hist_fe)]:
    print(f"{name:<20}{h['train_acc'][-1]:>10.2f}{h['val_acc'][-1]:>10.2f}{h['ti
```



Method	Train acc	Val acc	Time (s)
Fine-tuning	88.93	92.81	1296
Feature extractor	88.11	94.77	372

Explanation

- The model achieved **94.77% final validation accuracy**, with a best accuracy of 96.08% and 88.11% training accuracy.
- It reached 91.5% validation accuracy in the first epoch and 95.4% in the second, with a lower and more stable validation loss of around 0.15–0.17.
- Training took only **372 seconds**, approximately 3.5 times faster than fine-tuning, because only the final classification layer was updated.

Comparison and Discussion

Method	Train Accuracy	Validation Accuracy	Time (s)
Fine-tuning	88.93%	92.81%	1,296
Feature extraction	88.11%	94.77%	372

- Feature extraction achieved slightly higher validation accuracy with substantially less training time. Since the dataset contains only 244 training images, updating all pretrained weights may disrupt useful features and increase overfitting risk.
- The validation set contains only 153 images, so the accuracy difference of approximately 2 percentage points represents only about three images. Therefore, this difference is not strong evidence that one method is consistently better.
- Both models achieved higher validation than training accuracy because training images underwent random crops and flips, making the training task more challenging.
- Overall, feature extraction is a practical choice for this small dataset because it is faster and achieves comparable or better performance. Fine-tuning may become more beneficial with more training data or when the target task differs substantially from ImageNet.